CSP-J 2020 (Junior)

2020-11-07, 8:30–12:00 (3.5 hours)

This file is tailored to QOJ: every problem reads from standard input and writes to standard output (the original paper used the files `<code>.in` / `<code>.out` named after each problem).
Problems of the second round of the 2020 CCF Certified Software Professional (Junior group), in contest order. Every problem has time limit 1 s and memory limit 256 MB (as shown on QOJ), and is worth 100 points.

Problem 1: Excellent Split (优秀的拆分) QOJ 18071

Time limit: 1 s. Memory limit: 256 MB.

Problem description

In general, a positive integer can be split into a sum of several positive integers. For example, $1 = 1$, $10 = 1 + 2 + 3 + 4$, and so on.

A particular split of a positive integer n is called excellent if and only if, in this split, n is decomposed into several distinct positive-integer powers of 2. Note that a number x can be expressed as a positive-integer power of 2 if and only if x can be obtained by multiplying a positive number of 2s together.

For example, $10 = 8 + 2 = 2^3 + 2^1$ is an excellent split. However, $7 = 4 + 2 + 1 = 2^2 + 2^1 + 2^0$ is not an excellent split, because 1 is not a positive-integer power of 2.

Now, given a positive integer n, you need to determine whether, among all splits of this number, there exists an excellent split. If one exists, please give the concrete split.

Input

The input file has only one line, containing a positive integer n, the number to be examined.

Output

If, among all splits of this number, there exists an excellent split, then you need to output every number in this split from largest to smallest, with adjacent numbers separated by a single space. It can be proved that, once the order of the numbers in the split is fixed, the split is unique.

If no excellent split exists, output "`-1`" (without the quotation marks).

Sample 1 input

```
6
```

Sample 1 output

```
4 2
```

Explanation of sample 1

$6 = 4 + 2 = 2^2 + 2^1$ is an excellent split. Note that $6 = 2 + 2 + 2$ is not an excellent split, because the 3 numbers in the split do not satisfy the requirement that all numbers be pairwise distinct.

Sample 2 input

```
7
```

Sample 2 output

```
-1
```

Sample 3

Large sample, not printed here; provided as the downloadable files `power/power3.in` and `power/power3.ans`.

Constraints

For 20% of the data, $n \leq 10$.
For another 20% of the data, n is guaranteed to be odd.
For another 20% of the data, n is guaranteed to be a positive-integer power of 2.
For 80% of the data, $n \leq 1024$.
For 100% of the data, $1 \leq n \leq 1 \times 10^7$.

Problem 2: Live Awards (直播获奖) QOJ 18072

Time limit: 1 s. Memory limit: 256 MB.

Problem description

NOI2130 is about to be held. To make it more entertaining, CCF has decided to announce the contestants' scores one by one and to broadcast the real-time award cutoff score live. The award rate of this competition is $w\%$, that is, the lowest score among the contestants currently ranked in the top $w\%$ is the real-time cutoff score.

More specifically, if the scores of p contestants have been announced so far, then the current planned number of award winners is $\max(1, \lfloor p \times w\% \rfloor)$, where w is the award percentage, $\lfloor x \rfloor$ denotes x rounded down to an integer, and $\max(x, y)$ denotes the larger of x and y. If some contestants have equal scores, then all contestants tied at that score receive an award, so the actual number of award winners may be larger than planned.

As a technical staff member of the judging team, please help CCF write a live-broadcast program.

Input

Line 1 contains two positive integers n, w, the total number of contestants and the award rate, respectively.

Line 2 contains n non-negative integers, the contestants' scores in the order in which they are announced one by one.

Output

Only one line, containing n non-negative integers, which are, in order, the real-time award cutoff scores after each contestant's score is announced one by one. Adjacent integers are separated by a single space.

Sample 1 input

```
10 60
200 300 400 500 600 600 0 300 200 100
```

Sample 1 output

```
200 300 400 400 400 500 400 400 300 300
```

Explanation of sample 1

Number of contestants judged	1	2	3	4	5	6	7	8	9	10
Planned number of award winners	1	1	1	2	3	3	4	4	5	6

Scores of the judged contestants, sorted from high to low (the cutoff score is marked in bold)	200	300	400	500	600	600	600	600	600	600
		200	300	400	500	600	600	600	600	600
			200	300	400	500	500	500	500	500
				200	300	400	400	400	400	400
					200	300	300	300	300	300
						200	200	300	300	300
							0	200	200	200
								0	200	200
									0	100
										0

Note that after the score of the 9th contestant is announced, the planned number of award winners is 5, but because of ties, 6 contestants actually receive awards.

Sample 2 input

```
10 30
100 100 600 100 100 100 100 100 100 100
```

Sample 2 output

```
100 100 600 600 600 600 100 100 100 100
```

Sample 3

Large sample, not printed here; provided as the downloadable files `live/live3.in` and `live/live3.ans`.

Constraints

Test case number	n
1∼3	$= 10$
4∼6	$= 500$
7∼10	$= 2000$
11∼17	$= 10000$
18∼20	$= 100000$

For all test cases, every contestant's score is a non-negative integer not exceeding 600, and the award percentage w is a positive integer with $1 \leq w \leq 99$.

When computing the planned number of award winners, if a floating-point variable (such as `float` or `double` in C/C++, or `real`, `double`, `extended`, etc. in Pascal) is used to store the award ratio $w\%$, then the result of computing $5 \times 60\%$ may be 3.000001 or may be 2.999999, and the result after rounding down is not determined. Therefore, it is recommended to use only integer variables, so as to compute the exact value.

Problem 3: Expression (表达式) QOJ 18073

Time limit: 1 s. Memory limit: 256 MB.

Problem description

Little C is keen on studying mathematical logic. One day, he discovered a special kind of logical expression. In this kind of logical expression, all operands are variables, their values can only be 0 or 1, and the operations are carried out from left to right. If the expression contains parentheses, the value of the sub-expression inside the parentheses is computed first. In particular, this kind of expression has exactly the following operations:

1. AND: a `&` b. The value of this expression is 1 if and only if the values of both a and b are 1. In all other cases the value of this expression is 0.
2. OR: a `|` b. The value of this expression is 0 if and only if the values of both a and b are 0. In all other cases the value of this expression is 1.
3. NOT: `!` a. The value of this expression is 1 if and only if the value of a is 0. In all other cases the value of this expression is 0.

Little C wants to know: given a logical expression and the initial value of each of its operands, what is the value of the original expression after the value of one particular operand is negated?

To simplify the processing of the expression, we make the following conventions.

The expression is given in postfix notation. Postfix notation is defined as follows:

1. If E is an operand, then the postfix expression of E is E itself.
2. If E is an expression of the form E_1 *op* E_2, where *op* is any binary operator whose precedence is not higher than that of the operators outside parentheses in E_1 and E_2, then the postfix form of E is E_1' E_2' *op*, where E_1' and E_2' are the postfix forms of E_1 and E_2, respectively.
3. If E is an expression of the form (E_1), then the postfix form of E_1 is the postfix form of E.

Also, for convenience, in the input:

a) There is one space on each side of the AND operator (`&`), the OR operator (`|`) and the NOT operator (`!`), but there is no space at the end of the expression.

b) An operand consists of the lowercase letter `x` followed by a positive integer, the positive integer being the index of the variable. For example, `x10` denotes the variable x_{10} with index 10. It is guaranteed that every variable appears exactly once in the expression.

Input

The first line contains a string s, the expression described above.

The second line contains a positive integer n, the number of variables in the expression. The indices of the variables in the expression are $1, 2, \ldots, n$.

The third line contains n integers, the i-th of which is the initial value of the variable x_i.

The fourth line contains a positive integer q, the number of queries.

Each of the next q lines contains a positive integer, the index of the variable to be negated. Note that the modification in each query is temporary, that is, the modification in an earlier query has no effect on later queries.

It is guaranteed that the input expression is valid. The initial value of each variable is 0 or 1.

Output

The output consists of q lines, each containing a 0 or a 1, the value of the expression under the corresponding query.

Sample 1 input

```
x1 x2 & x3 |
3
101
3
1
2
3
```

Sample 1 output

```
1
1
0
```

Explanation of sample 1

The infix form of this postfix expression is $(x_1$ `&` $x_2)$ `|` x_3.

For the first query, the value of x_1 is negated. Then the values of the three operands are, in order, 0, 0, 1. The value of the original expression is $(0$ `&` $0)$ `|` $1 = 1$.

For the second query, the value of x_2 is negated. Then the values of the three operands are, in order, 1, 1, 1. The value of the original expression is $(1$ `&` $1)$ `|` $1 = 1$.

For the third query, the value of x_3 is negated. Then the values of the three operands are, in order, 1, 0, 0. The value of the original expression is $(1$ `&` $0)$ `|` $0 = 0$.

Sample 2 input

```
x1 ! x2 x4 | x3 x5 ! & & ! &
5
01011
3
1
3
5
```

Sample 2 output

```
0
1
1
```

Explanation of sample 2

The infix form of this expression is (`!` x_1) `&` (`!` ((x_2 `|` x_4) `&` (x_3 `&` (`!` x_5)))).

Sample 3

Large sample, not printed here; provided as the downloadable files `expr/expr3.in` and `expr/expr3.ans`.

Constraints

For 20% of the data, the expression contains only AND operations (`&`) or only OR operations (`|`).
For another 30% of the data, $|s| \leq 1000$, $q \leq 1000$, $n \leq 1000$.
For another 20% of the data, the initial values of the variables are all 0 or all 1.
For 100% of the data, $1 \leq |s| \leq 1 \times 10^6$, $1 \leq q \leq 1 \times 10^5$, $2 \leq n \leq 1 \times 10^5$.
Here $|s|$ denotes the length of the string s.

Problem 4: Taking Numbers on a Grid (方格取数) QOJ 18074

Time limit: 1 s. Memory limit: 256 MB.

Problem description

There is an $n \times m$ grid, and every cell contains an integer. A little bear wants to walk from the top-left corner of the grid to the bottom-right corner. In each step it may move only one cell up, down or right, it may not pass through a cell it has already visited, and it may not leave the grid. The bear takes the integers in all cells it passes through. Find the maximum possible sum of the integers it takes.

Input

Line 1 contains two positive integers n, m.

Each of the next n lines contains m integers, the integers in the cells, in order.

Output

One integer, the maximum possible sum of the integers the bear can take.

Sample 1 input

```
3 4
1 -1 3 2
2 -1 4 -1
-2 2 -3 -1
```

Sample 1 output

```
9
```

Explanation of sample 1

In the figures below, the cells on the path are shown in bold and the arrows show the direction of each step.

$$\begin{array}{ccccccc} 1 & & -1 & & 3 & \rightarrow & 2 \\ \downarrow & & & & \uparrow & & \downarrow \\ 2 & \rightarrow & \mathbf{-1} & \rightarrow & 4 & & \mathbf{-1} \\ & & & & & & \downarrow \\ -2 & & 2 & & -3 & & \mathbf{-1} \end{array}$$

With the walk shown above, the sum of the numbers taken is $1+2+(-1)+4+3+2+(-1)+(-1) = 9$, and it can be proved that this is the maximum.

$$\begin{array}{ccccccc} 1 & & -1 & & 3 & \rightarrow & 2 \\ \downarrow & & & & \uparrow & & \downarrow \\ 2 & \rightarrow & \mathbf{-1} & \rightarrow & 4 & & \mathbf{-1} \\ & & \downarrow\uparrow & & & & \downarrow \\ -2 & & 2 & & -3 & & \mathbf{-1} \end{array}$$

Note that the walk shown above is invalid, because the cell in row 2, column 2 is visited twice, and according to the problem statement a cell that has already been visited may not be passed through again.

$$\begin{array}{cccccccc} 1 & & -1 & & 3 & \rightarrow & 2 \\ \downarrow & & & & \uparrow & & \\ 2 & \rightarrow & \mathbf{-1} & \rightarrow & 4 & & -1 \\ & & & & & & \\ -2 & & 2 & & -3 & & -1 \end{array}$$

Moreover, the walk shown above is also invalid, because it does not reach the end point at the bottom-right corner.

Sample 2 input

```
2 5
-1 -1 -3 -2 -7
-2 -1 -4 -1 -2
```

Sample 2 output

```
-10
```

Explanation of sample 2

$$\begin{array}{ccccccccc} \mathbf{-1} & \rightarrow & \mathbf{-1} & \rightarrow & \mathbf{-3} & \rightarrow & \mathbf{-2} & & -7 \\ & & & & & & \downarrow & & \\ -2 & & -1 & & -4 & & \mathbf{-1} & \rightarrow & \mathbf{-2} \end{array}$$

With the walk shown above, the sum of the numbers taken is $(-1) + (-1) + (-3) + (-2) + (-1) + (-2) = -10$, and it can be proved that this is the maximum. Therefore, please note that the maximum sum of the numbers taken may also be negative.

Sample 3

Large sample, not printed here; provided as the downloadable files `number/number3.in` and `number/number3.ans`.

Constraints

For 20% of the data, $n, m \leq 5$.

For 40% of the data, $n, m \leq 50$.

For 70% of the data, $n, m \leq 300$.

For 100% of the data, $1 \leq n, m \leq 1000$. The absolute value of every integer in the grid does not exceed 10^4.

Source

The statements were taken from the official CSP-J 2020 problem PDFs served by QOJ (problem pages, “Statement” tab):

- Problem 1: `https://qoj.ac/problem/18071`
- Problem 2: `https://qoj.ac/problem/18072`
- Problem 3: `https://qoj.ac/problem/18073`
- Problem 4: `https://qoj.ac/problem/18074`

Contest page: `https://qoj.ac/contest/2860` (CSP-J 2020).